

1. Write a program to perform the following
 - An empty list
 - A list with one element
 - A list with all identical elements
 - A list with negative numbers

1.

```
def simple_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(i + 1, n):
            if arr[i] > arr[j]:
                arr[i], arr[j] = arr[j], arr[i]

    return arr
print(simple_sort([]))
print(simple_sort([1]))
print(simple_sort([7, 7, 7, 7]))
print(simple_sort([-5, -1, -3, -2, -4]))
```

```
... []
    [1]
    [7, 7, 7, 7]
    [-5, -4, -3, -2, -1]
```

2. Describe the Selection Sort algorithm's process of sorting an array. Selection Sort works by dividing the array into a sorted and an unsorted region. Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order, ensuring clarity in your explanation of the algorithm's mechanics and effectiveness.

2

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
print(selection_sort([5,2,9,1,5,6]))
print(selection_sort([10,8,6,4,2]))
print(selection_sort([1,2,3,4,5]))
```

```
[1, 2, 5, 5, 6, 9]
[2, 4, 6, 8, 10]
[1, 2, 3, 4, 5]
```

3. Write code to modify bubble_sort function to stop early if the list becomes sorted before all passes are completed.

3.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
print(bubble_sort([64,25,12,22,11]))
print(bubble_sort([29,10,14,37,13]))
print(bubble_sort([3,5,2,1,4]))
print(bubble_sort([1,2,3,4,5]))
print(bubble_sort([5,4,3,2,1]))
```

```
... [11, 12, 22, 25, 64]
     [10, 13, 14, 29, 37]
     [1, 2, 3, 4, 5]
     [1, 2, 3, 4, 5]
     [1, 2, 3, 4, 5]
```

4. Write code for Insertion Sort that manages arrays with duplicate elements during the sorting process. Ensure the algorithm's behavior when encountering duplicate values, including whether it preserves the relative order of duplicates and how it affects the overall sorting outcome.

4.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
print(insertion_sort([3,1,4,1,5,9,2,6,5,3]))
print(insertion_sort([5,5,5,5,5]))
print(insertion_sort([2,3,1,3,2,1,1,3]))
```

```
[1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
[5, 5, 5, 5, 5]
[1, 1, 1, 2, 2, 3, 3, 3]
```

5. Given an array `arr` of positive integers sorted in a strictly increasing order, and an integer `k`. return the `k`th positive integer that is missing from this array.

5.

```
def findKthPositive(arr, k):
    missing = []
    num = 1
    i = 0
    while len(missing) < k:
        if i < len(arr) and arr[i] == num:
            i += 1
        else:
            missing.append(num)
            num += 1
    return missing[-1]
print(findKthPositive([2,3,4,7,11],5))
print(findKthPositive([1,2,3,4],2))
```

9
6

6. A peak element is an element that is strictly greater than its neighbors. Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks. You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array. You must write an algorithm that runs in $O(\log n)$ time.

6.

```
def findPeak(nums):
    left = 0
    right = len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] < nums[mid + 1]:
            left = mid + 1
        else:
            right = mid
    return left
print(findPeak([1,2,3,1]))
print(findPeak([1,2,1,3,5,6,4]))
```

... 2
5

7. Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or -1 if `needle` is not part of `haystack`.

Example 1:

Input: haystack = "sadbutsad", needle = "sad"

Output: 0

Explanation: "sad" occurs at index 0 and 6.

The first occurrence is at index 0, so we return 0.

Example 2:

Input: haystack = "leetcode", needle = "leeto"

Output: -1

Explanation: "leeto" did not occur in "leetcode", so we return -1.

7.

```
def strStr(haystack, needle):
    n = len(haystack)
    m = len(needle)
    for i in range(n - m + 1):
        if haystack[i:i+m] == needle:
            return i
    return -1
print(strStr("sadbutsad", "sad"))
print(strStr("leetcode", "leeto"))
```

0
-1

8. Given an array of string words, return all strings in words that is a substring of another word. You can return the answer in any order. A substring is a contiguous sequence of characters within a string

8.

```
def stringMatching(words):
    result = []
    for i in range(len(words)):
        for j in range(len(words)):
            if i != j and words[i] in words[j]:
                result.append(words[i])
                break
    return result
print(stringMatching(["mass", "as", "hero", "superhero"]))
print(stringMatching(["leetcode", "et", "code"]))
print(stringMatching(["blue", "green", "bu"]))
```

```
... ['as', 'hero']
    ['et', 'code']
    []
```

9. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

9.

```
import math
def closest_pair(points):
    min_dist = float("inf")
    pair = ()
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = math.sqrt((points[i][0]-points[j][0])**2 +
                          (points[i][1]-points[j][1])**2)
            if d < min_dist:
                min_dist = d
                pair = (points[i], points[j])
    return pair, min_dist
points = [(1,2), (4,5), (7,8), (3,1)]
pair, dist = closest_pair(points)
print("Closest Pair:", pair)
print("Minimum Distance:", dist)
```

```
Closest Pair: ((1, 2), (3, 1))
Minimum Distance: 2.23606797749979
```

10. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7 (6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the sameline?

10.

```
def orientation(p, q, r):
    return (q[0]-p[0])*(r[1]-p[1]) - (q[1]-p[1])*(r[0]-p[0])
def convex_hull(points):
    n = len(points)
    hull = []
    for i in range(n):
        for j in range(i + 1, n):
            left = right = 0
            for k in range(n):
                if k == i or k == j:
                    continue
                val = orientation(points[i], points[j], points[k])
                if val > 0:
                    left += 1
                elif val < 0:
                    right += 1
            if left == 0 or right == 0:
                if points[i] not in hull:
                    hull.append(points[i])
                if points[j] not in hull:
                    hull.append(points[j])
    return hull
points = [
    (10,0), (11,5), (5,3), (9,3.5),
    (15,3), (12.5,7), (6,6.5), (7.5,4.5)
]
print("Convex Hull:")
print(convex_hull(points))
```

```
*** Convex Hull:
[(10, 0), (5, 3), (15, 3), (6, 6.5), (12.5, 7)]
```

11. Write a program that finds the convex hull of a set of 2D points using the brute force approach.

11.

```
def orientation(p, q, r):
    return (q[0]-p[0])*(r[1]-p[1]) - (q[1]-p[1])*(r[0]-p[0])
def convex_hull(points):
    hull = []
    n = len(points)
    for i in range(n):
        for j in range(i+1, n):
            left = right = 0
            for k in range(n):
                if k == i or k == j:
                    continue
                val = orientation(points[i], points[j], points[k])
                if val > 0:
                    left += 1
                elif val < 0:
                    right += 1
            if left == 0 or right == 0:
                if points[i] not in hull:
                    hull.append(points[i])
                if points[j] not in hull:
                    hull.append(points[j])
    return hull
points = [(1,1), (4,6), (8,1), (0,0), (3,3)]
print("Convex Hull:", convex_hull(points))
```

Convex Hull: [(4, 6), (8, 1), (0, 0)]

12. You are given a list of cities represented by their coordinates. Develop a program that utilizes exhaustive search to solve the TSP. The program should:

1.).
- 12.

```
▶ from itertools import permutations
import math
def distance(a, b):
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)
def tsp(cities):
    start = cities[0]
    others = cities[1:]
    min_dist = float("inf")
    best_path = None
    for path in permutations(others):
        route = [start] + list(path) + [start]
        total = 0
        for i in range(len(route)-1):
            total += distance(route[i], route[i+1])
        if total < min_dist:
            min_dist = total
            best_path = route
    return min_dist, best_path
cities = [(1,2), (4,5), (7,1), (3,6)]
d, path = tsp(cities)
print("Shortest Distance:", d)
print("Shortest Path:", path)
```

```
*** Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]
```

13. You are given a cost matrix where each element $\text{cost}[i][j]$ represents the cost of assigning worker i to task j . Develop a program that utilizes exhaustive search to solve the assignment

problem. The program should Define a function `total_cost(assignment, cost_matrix)` that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function `assignment_problem(cost_matrix)` that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions).

13.

```
▶ from itertools import permutations
def assignment_problem(cost):
    n = len(cost)
    min_cost = float("inf")
    best = None
    for p in permutations(range(n)):
        total = 0
        for i in range(n):
            total += cost[i][p[i]]
        if total < min_cost:
            min_cost = total
            best = p

    return best, min_cost

cost = [
    [3,10,7],
    [8,5,12],
    [4,6,9]
]
assign, total = assignment_problem(cost)
print("Assignment:", assign)
print("Minimum Cost:", total)
```

```
... Assignment: (2, 1, 0)
Minimum Cost: 16
```

14. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:

14.

```
▶ from itertools import combinations
def knapsack(weights, values, capacity):
    n = len(weights)
    best_value = 0
    best_items = []
    for r in range(n+1):
        for subset in combinations(range(n), r):
            total_weight = 0
            total_value = 0
            for i in subset:
                total_weight += weights[i]
                total_value += values[i]
            if total_weight <= capacity and total_value > best_value:
                best_value = total_value
                best_items = subset
    return best_items, best_value
weights = [2,3,1]
values = [4,5,3]
capacity = 4
items, value = knapsack(weights, values, capacity)
print("Selected Items:", items)
print("Maximum Value:", value)
```

```
... Selected Items: (1, 2)
Maximum Value: 8
```